



CS401 MPP

W3D5 Overview

Java 8 Best Practices

Overview of Today

- Unit Testing Stream Pipelines
- Introduction to Annotations
- Try with Resources
- Handling Exceptions in Streams
- Introduction to Threads
- Parallel Processing

Wholeness Statement

- Best practices in the world of OO programming are a way of ensuring quality in code.
 - Code that adheres to best practices tends to be easier to understand, easier to maintain, more capable of adapting in the face of changing requirements and new feature requests, and more reusable for other projects.
- This simple theme is reflected in individual life.
 - There are laws governing life, both physical laws and laws pertaining to all kinds of relationships and interactions.
 - When life flows in accordance with the laws of nature, life is supported for success and fulfillment.
 - When awareness becomes established at its deepest level, actions and behavior springing from this profound quality of awareness spontaneously are in accord with the laws of nature.



CS401 MPP

Java 8 Best Practices

Unit Testing Stream Pipelines

Unit Testing Stream Pipelines

- How do we unit test stream pipelines?
 - Stream pipelines, with lambdas mixed in, form a single unit
 - it is not obvious how to unit test the pieces of the pipeline
- Two guidelines:
 - If the pipeline is simple enough, we can name it as an expression and unit test it directly.
 - If the pipeline is more complex, pieces of pipeline can be called from support methods, and the support methods can be unit-tested.

Testing Simple Expressions

- We want to test the following pipeline:

```
Function<List<String>, List<String>> allToUpperCase =  
    words -> words.stream()  
        .map(word -> word.toUpperCase())  
        .collect(Collectors.toList());
```

- We can do ordinary unit testing of **allToUpperCase**
 - Assuming it is stored in the Lambda-Library **Testing**

```
@Test  
public void multipleWordsToUppercase() {  
    List<String> input = Arrays.asList("a", "b", "hello");  
    List<String> result = Testing.allToUpperCase.apply(input);  
    assertEquals(Arrays.asList("A", "B", "HELLO"), result);  
}
```

Testing Complex Expressions

- Sort Employees first by name (ASC), then by salary (DESC)

//difficult to test

```
static Function<List<Employee>, List<Employee>> employeeSorter =  
    list -> list.stream()  
        .sorted(Comparator.comparing((Employee e) -> e.getName())  
                .thenComparing((Employee e) -> e.getSalary(),  
                               Comparator.reverseOrder()))  
        .collect(Collectors.toList());
```

Break into pieces and test each piece

- Need to test if Employees are compared correctly
- Can be done by taking the expression that defines the Comparator and putting it into its own companion (lambda-library) class
- Then use a method reference to put it back into the whole

Refactoring the Pipeline

Another Lambda-Library

- Put the **Comparator** into LibraryCompanion:

```
static Function<List<Employee>, List<Employee>> employeeSorter =  
    list -> list.stream()  
        .sorted(LibraryCompanion::compareEmps)  
        .collect(Collectors.toList());
```

```
public class LibraryCompanion {  
    static Comparator<Employee> empComp  
        = Comparator.comparing((Employee e) -> e.getName())  
            .thenComparing((Employee e) -> e.getSalary(),  
                Comparator.reverseOrder());  
    public static int compareEmps(Employee e1, Employee e2) {  
        return empComp.compare(e1, e2);  
    }  
}
```

Now we can test the comparison!

Auxiliary helper method to use the lambda

Testing LibraryCompanion

```
//verify that jim comes before joe  
assert(LibraryCompanion.compareEmps(jim, joe1) < 0);  
  
//verify that joe2 comes before joe1  
assert(LibraryCompanion.compareEmps(joe2, joe1) < 0);
```

Best Practice

- The best practice for Complex Expressions :
 - The Lambdas of the complex expression should go into a Lambda library along with helper methods
 - The original location then uses method references (to the helpers) to use the lambdas
 - We can simply test the helper like any other method

Exercise 10.1

- You are developing unit tests and need to decide how to test the lambda expression shown below
 - Should you treat it as a simple or a complex expression?
 - See lesson10.exercise_1 in the InClassExercises
 - Create a unit test in the TestLambda class

```
//A list of Customers whose account balance is > 50, sorted by customer's last name
public static List<Customer> specialAccounts(List<Account> accounts) {
    return accounts.stream().filter(a -> a.getBalance() > 50)
        .map((Account a) -> a.getCustomer())
        .sorted(Comparator.comparing((Customer c) -> c.getLastName()))
        .collect(Collectors.toList());
}
```

Hint: there is no need to test getters or setters



CS401 MPP

Java 8 Best Practices

Introduction to Annotations

What are Annotations?

- We have seen them in various contexts already:
 - `@Test` - JUnit
 - `@Override` – to indicate that a method is being overridden
 - `@FunctionalInterface` – to indicate that an interface is functional and may be used with lambdas
 - `@Deprecated` – to discourage use of a method or class
 - `@SuppressWarnings` – to hide warning messages of various kinds

What are Annotations?

- Annotations are tags inserted into source code so that tools can process them
 - These tools can operate on the source level
 - Or at the bytecode level, processing class files which still contain annotations
- To benefit from annotations that you create, you need to select a processing tool
 - You need to use annotations that your processing tool understand
 - Then apply the processing tool to your code
- Annotations can be applied to a class, a method, a variable
 - Essentially anywhere qualifiers like public and static may be used

The tool for almost all the annotations on the previous slide was the compiler
Only @Test has a different tool: JUnit

Annotations

- Annotations may have zero or more elements.
 - Here is an example of a user-defined annotation that has two elements, **assignedTo** and **severity**
 - This annotation can be applied at the class level

```
@Bugreport(assignedTo="Harry", severity=10)
```

- Like any user-defined annotation
 - it will require an external tool to process it

Optional Compact Syntax

- There is a more compact syntax
 - Can only be used when an annotation has just one element and its name is “value”

```
@SuppressWarnings("unchecked")
```

```
@SuppressWarnings(value = "unchecked")
```

These two do the same thing

Repeating Annotations

- The same annotation multiple times on the same element is called a repeating annotation:

```
@Schedule(dayOfMonth="last")  
@Schedule(dayOfWeek="Fri", hour="23")  
public void doPeriodicCleanup() { }
```

Note that not all annotations are repeating

Only those that are created with `@Repeatable`

- Repeating annotations were added to Java 8
 - Earlier releases did not support this feature

User Defined Annotations

- The `@interface` keyword is the way the Java compiler knows you are creating an annotation
 - Such “classes” extend the Annotation interface

```
@Retention(RetentionPolicy.RUNTIME)  
@Target(ElementType.TYPE)  
public @interface Bugreport {  
    String assignedTo() default "none";  
    int severity() default 0;  
}
```

Note that the code shows how the elements "assigned to" and "severity" are defined.

@Retention

- **@Retention** specifies how the annotation is stored:
 - **RetentionPolicy.SOURCE**
 - The annotation is stored only in the source and is ignored by the compiler.
 - **RetentionPolicy.CLASS**
 - The annotation is retained by the compiler at compile time
 - But is ignored by the Java Virtual Machine (not available at runtime)
 - **RetentionPolicy.RUNTIME**
 - The marked annotation is retained by the JVM
 - It can be used by the runtime environment.

@Target

- **@Target** restrict the kind of Java elements the annotation can be applied to
 - **ElementType.ANNOTATION_TYPE** can be applied to an annotation type
 - **ElementType.CONSTRUCTOR** can be applied to a constructor
 - **ElementType.FIELD** can be applied to a field or property
 - **ElementType.LOCAL_VARIABLE** can be applied to a local variable
 - **ElementType.METHOD** can be applied to a method-level annotation
 - **ElementType.PACKAGE** can be applied to a package declaration
 - **ElementType.PARAMETER** can be applied to the parameters of a method
 - **ElementType.TYPE** can be applied to a class or interface

Exercise 10.2

- Work with the Bug Report code
 - Available in lesson10.exercise_2 of InClassExercises
- A class **StillMoreBadCode** has been added
 - Add to this annotation values for "**assigned to**" and "**severity**"
 - Then integrate this new class into the **AnnotationInfo** class and run it
 - You should now see info about **StillMoreBadCode** in the output report



CS401 MPP

Java 8 Best Practices

Try with Resources

Using Resources

- You Wish to Use Resources Carefully.
 - When your application uses external resources,
 - like files or a database connection,
 - it is important to close the connections once your finished with them
 - Typically, using these connections involves checked exceptions
 - but whether or not an exception is thrown
 - your application must disconnect from the resource
 - or there can be a memory leak – causing memory to fill up

Clean up inside Finally

- Clean-up in a finally block:
 - The usual way to clean up resources is in a finally block
 - It will execute whether or not an exception is thrown.
- Complication:
 - What if closing a resource is also capable of throwing a checked exception?
 - How should this be handled?

See: `lesson10.lecture.trickycatch1`

Example

```
public void handleFile(File f) throws IOException {
    FileReader fileReader = null;
    BufferedReader buf = null;
    try{
        fileReader = new FileReader(f);
        buf = new BufferedReader(fileReader);
        String line = buf.readLine();
        System.out.println("Line from file: " + line);
    } catch(IOException e) {
        log.config("Caught 1st IOException: " + e.getMessage());
        throw(e);
    } finally {
        try {
            if(buf != null) buf.close();
            if(fileReader != null) fileReader.close();
            throw new IOException("Caught 2nd IOException: error closing readers");
        } catch(IOException e) {
            log.fine(e.getMessage());
            throw(e);
        }
    }
}
```

Before Java 7, this was very common

Code Problems

1. It's messy and it's easy to implement it incorrectly

2. A More serious problem:

- After the `BufferedReader` object is created, a call to `readLine` causes an `IOException` to be thrown.
- In the finally clause, an attempt is made to close the `BufferedReader`, but the `close()` operation also throws an `IOException`
- Only one exception can be thrown, and the way Java does it is to throw the close operation exception (the one that we don't care about).

In the demo code, the message displayed is
"Caught 2nd `IOException`: error closing reader."
The `IOException` indicating a failure to execute `readLine()` is lost

See: `lesson10.lecture.trickycatch1`

Try with Resources

- Since Java 7 there is an alternative solution:

```
public void handleFile(File f) {  
    //try with resources construct  
    try (BufferedReader buf = new BufferedReader(new FileReader(f))) {  
        String line = buf.readLine();  
        System.out.println(line);  
    } catch (IOException e) {  
        log.warning("Main exception: " + e.getMessage());  
        List<Throwable> suppressed = Arrays.asList(e.getSuppressed());  
        suppressed.forEach(except -> log.warning("Suppressed message: "  
                                                + except.getMessage()));  
    }  
}
```

See: `lesson10.lecture.trickycatch4_trywithres.MyClass`

Notes

- The resources in the "try" clause will be closed at the completion of the try block
 - In our example that's the **BufferedReader**
- If an exception is thrown during execution of try
 - And an error occurs in closing these resources, then the close exceptions will automatically be appended to the main exception as a suppressed exception
- If no main exception occurs
 - But a close exception does occur, the close exception is thrown in the usual way
- You can read suppressed exceptions using the **getSuppressed()** method
 - It returns a list of **Throwables** which can be read and handled if desired
 - But the main exception is the one that is thrown and caught

More About Resources

- A resource is an object that must be closed when we're finished with it
 - The try-with-resources statement ensures that each resource is closed
- Another example of a resource that can be managed with try-with-resources is a Database (JDBC) Connection object
 - Handling exceptions and closing the connection in the right way and in the right sequence used to be error-prone
 - Using try-with-resources, it is straightforward to write code in the correct way

See: `lesson10.lecture.jdbc.read_trywithres`

To run the code, you need to set up JavaDB.
See `setup/setting-up-javadb`

Try with Resources can use:

- Any object that implements `java.lang.AutoClosable`

`AbstractInterruptibleChannel, AbstractSelectableChannel, AbstractSelector, AsynchronousFileChannel, AsynchronousServerSocketChannel, AsynchronousSocketChannel, AudioInputStream, BufferedInputStream, BufferedOutputStream, BufferedReader, BufferedWriter, ByteArrayInputStream, ByteArrayOutputStream, CharArrayReader, CharArrayWriter, CheckedInputStream, CheckedOutputStream, CipherInputStream, CipherOutputStream, DatagramChannel, DatagramSocket, DataInputStream, DataOutputStream, DeflaterInputStream, DeflaterOutputStream, DigestInputStream, DigestOutputStream, FileCacheImageInputStream, FileCacheImageOutputStream, FileChannel, FileImageInputStream, FileImageOutputStream, FileInputStream, FileLock, FileOutputStream, FileReader, FileSystem, FileWriter, FilterInputStream, FilterOutputStream, FilterReader, FilterWriter, Formatter, ForwardingJavaFileManager, GZIPInputStream, GZIPOutputStream, ImageInputStreamImpl, ImageOutputStreamImpl,InflaterInputStream, InflaterOutputStream, InputStream, InputStream, InputStream, InputStreamReader, JarFile, JarInputStream, JarOutputStream, LineNumberInputStream, LineNumberReader, LogStream, MemoryCacheImageInputStream, MemoryCacheImageOutputStream, MLet, MulticastSocket, ObjectInputStream, ObjectOutputStream, OutputStream, OutputStream, OutputStream, OutputStreamWriter, Pipe.SinkChannel, Pipe.SourceChannel, PipedInputStream, PipedOutputStream, PipedReader, PipedWriter, PrintStream, PrintWriter, PrivateMLet, ProgressMonitorInputStream, PushbackInputStream, PushbackReader, RandomAccessFile, Reader, RMIClientConnectionImpl, RMIClientConnectionImpl_Stub, RMIClientConnector, RMIIIOpsServerImpl, RMIIJRMPServerImpl, RMIServerImpl, Scanner, SelectableChannel, Selector, SequenceInputStream, ServerSocket, ServerSocketChannel, Socket, SocketChannel, SSLServerSocket, SSLSocket, StringBufferInputStream, StringReader, StringWriter, URLClassLoader, Writer, XMLDecoder, XMLEncoder, ZipFile, ZipInputStream, ZipOutputStream`

These all implement
AutoClosable

Exercise 10.3

- Use try-with-resources to rewrite the class **OldFileIO**
 - It reads a file word_test.txt using the pre-Java 7 approach
 - See lesson10.exercise_3 of the InClassExercises
1. Include a catch to catch the primary **IOException** (if there is one)
 2. In your catch, loop through and print any suppressed exceptions that may be thrown (as demonstrated on slide 6)

Main Point 1

- Java's new try-with-resources handles the complex logic behind exception-handling when the process requires that a resource be "closed" after its use.
 - Earlier versions of Java required developers to handle this situation with an error-prone imperative programming style
 - Java's new approach is declarative: The developer just tells the runtime which resources will be used and the runtime takes care of the close operations, including exception-handling.
- This effortless approach to handling code has a parallel in human life:
 - When individual awareness is sufficiently attuned to this deeper levels of consciousness, very little effort is required to achieve significant success.



CS401 MPP

Java 8 Best Practices

Handling Exceptions in Streams Pipelines

Exceptions in Stream Pipelines

- For ordinary functional expressions exception-handling can often be done in the usual way
 - Even when composed in a pipeline

```
public static void printFile(File f) {  
    try (BufferedReader reader = new BufferedReader(new FileReader(f))) {  
        System.out.println(reader.lines().collect(Collectors.joining(", ")));  
    } catch (IOException e) {  
        System.out.println("Error printing file: " + e.getMessage());  
    }  
}
```

The Problem

- Stream operations that require a functional interface whose unique method does not have a throws clause make exception-handling more difficult
 - Like map and filter, which use Function and Predicate

```
public List<String> getCanonicalPaths(String[] dirs) throws IOException {  
    return Stream.of(dirs)  
        .map(path -> new File(path).getCanonicalPath())  
        .collect(Collectors.toList());  
}
```

Compiler error: Unhandled IOException

```
interface Function<T, R> {  
    R apply(T t);  
}
```

No throws!

Solutions

- There are a couple of options
 - Write try / catch inside your lambdas
 - Write new functional interfaces that do throw
- The best solution is to make helpers methods to automate conversion to these new interfaces

See: `lesson10.lecture.exceptions2`

Example

```
@FunctionalInterface
public interface FunctionWithException<T, R> {
    R apply(T t) throws Exception;

    public static <T,R> Function<T,R> helper(FunctionWithException<T,R> f) {
        return x -> {
            try {
                return f.apply(x);
            } catch (Exception e) {
                throw new RuntimeException(e);
            }
        };
    }
}
```

```
public List<String> getCanonicalPaths(String[] dirs) {
    return Stream.of(dirs).map(path -> helper((String p) ->
        new File(p).getCanonicalPath()).apply(path))
        .map(Object::toString).collect(Collectors.toList());
}
```

helper is declared as a static import
to improve readability



CS401 MPP

Java 8 Best Practices

Introduction to Threads

Processes

- A process in Java is an instance of a Java program that is being executed
 - It contains the program code and its current activity
- A process has a self-contained execution environment
 - A process generally has a complete, private set of basic run-time resources; in particular, each process has its own memory space

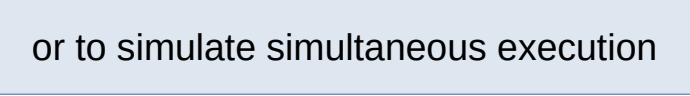
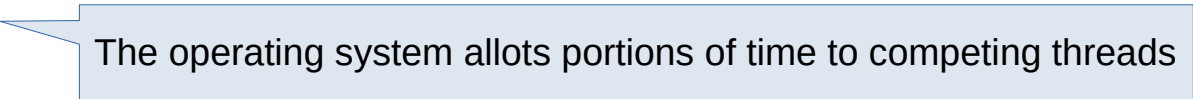
Threads

- A thread is a component of a process
 - Multiple threads can exist within the same process
 - They execute concurrently (one starting before others finish)
 - They share (heap) memory (and other resources)
- While different processes do not share these resources
 - In particular, the threads of a process share the values of its (heap) variables at any given moment

At Least One

- Every process has at least one thread
 - The main thread (the main method of a Java program starts up the main thread)
 - Other threads can be created from the main thread

Why More Than One?

- Multiple threads are typically invoked to perform multiple tasks simultaneously  or to simulate simultaneous execution
 - In a multiprocessor environment, different threads can access different processors
 - In a single processor environment, multiple threads can appear to work simultaneously by virtue of time-slicing  The operating system allots portions of time to competing threads

Multi Thread Examples

- One thread keeps a UI active while another thread performs a computation or accesses a database
- Divide up a long computation into pieces and let each thread compute values for one piece, then combine the results (computing in parallel)
- Web servers typically handle client requests on separate threads; in this way, clients can be served simultaneously

Creating Threads

- Put the code for a new thread in the **run()** method of a class that implements **Runnable**

```
interface Runnable {  
    void run() ;  
}
```

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Running a thread!");  
    }  
}
```

Creating Threads

- The thread is spawned when an instance of your **Runnable** class is used as an argument to the **Thread** constructor, and the **start()** method is called on the Thread instance

```
MyRunnable myRunnable = new MyRunnable();  
Thread t = new Thread(myRunnable);  
t.start();
```

Testing Singleton, One at a Time

```
public class Singleton {  
    private static Singleton instance;  
    public static int counter = 0;  
    private Singleton() {  
        incrementCounter();  
    }  
    public static Singleton getInstance() {  
        if(instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
    private static void incrementCounter() {  
        counter++;  
    }  
}
```

As expected
only 1 instance
is ever created

```
public class SingleThreadedTest2 {  
    public static void main(String[] args) {  
        for(int i = 0; i < 10; ++i) {  
            createAndStartThread();  
            System.out.println("Num instances: " + Singleton.counter);  
        }  
    }  
    public static void createAndStartThread() {  
        Runnable r = () -> {  
            for(int i = 0; i < 1000; ++i) {  
                Singleton.getInstance();  
            }  
        };  
        new Thread(r).start();  
        try {  
            Thread.sleep(10);  
        } catch (InterruptedException e) {}  
    }  
}
```

We have put each thread to sleep for 10 milliseconds at the start of its existence. Otherwise the first 1 or 2 calls of createAndStart will record 0 instances. This is because the change made by a thread may not be visible to the main thread immediately (this is most likely because processor memory is much faster than the RAM where the counter data is stored).

Testing Singleton, Multiple Threads

```
public class Singleton {  
    private static Singleton instance;  
    public static int counter = 0;  
    private Singleton() {  
        incrementCounter();  
    }  
    public static Singleton getInstance() {  
        if(instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
    private static void incrementCounter() {  
        counter++;  
    }  
}
```

```
public class MultiThreadedTest {  
    public static void main(String[] args) {  
        for(int i = 0; i < 10; ++i) {  
            multipleCalls();  
            System.out.println("Num instances: " + Singleton.counter);  
        }  
    }  
    public static void multipleCalls() {  
        Runnable r = () -> {  
            for(int i = 0; i < 5000; ++i) {  
                Singleton.getInstance();  
            }  
        };  
        for(int i = 0; i < 1000; ++i) {  
            new Thread(r).start();  
        }  
    }  
}
```

The results show that competing threads are creating multiple instances of the Singleton class. The test "instance == null" is being interrupted (single core) or just true at the same time (multi-core) so that it appears / is true to more than one thread and so the constructor is called multiple times.

Race Conditions / Thread Safety

- Two or more threads have access to the same object and each modifies the state of the object
 - This situation is called a race condition
 - It arises when threads interfere with each other
 - As we saw on our previous slide
- Code is said to be thread-safe
 - if it is guaranteed to be free of race conditions when accessed by multiple threads simultaneously.



Our Singleton was clearly
not Thread-safe!

Atomic Operations

- Certain operations are guaranteed to be atomic
 - Cannot be interrupted (single core machine)
 - Only one CPU can execute it at the same time
- The Java Specification guarantees that reading and writing memory addresses (and most) primitives are atomic operations
 - Long and Double are 64bits: read/writes not atomic on 32bit machines
 - `int x=2` is always atomic
 - `x++` is not atomic (in Java) it becomes `x = x + 1`

Synchnorized

- We can force threads to access the **getInstance** method of **Singleton** one at a time
 - By labeling **getInstance** with **synchronized**
 - This is called serialized access
- When a method is **synchronized**
 - A thread must acquire the lock for the instance of the object that the method is running on
 - In order for the thread to execute the method

Lock acquisition is an atomic operation

Object Locks

- Each Java object has an *intrinsic* lock
 - A thread gains access to this lock when it calls the method, as long as no other thread has the lock
- Once a thread has finished the synchronized method
 - the lock becomes available again and the next eligible thread (determined by the OS using thread priorities and other factors) acquires the lock

Serialized Access with Synchronized

```
public class Singleton2 {  
    private static Singleton2 instance;  
    public static int counter = 0;  
    private Singleton2() {  
        incrementCounter();  
    }  
    public static Singleton2 getInstance() {  
        if(instance == null) {  
            instance = new Singleton2();  
        }  
        return instance;  
    }  
    /* Guarantees proper count of instances */  
    synchronized private static void incrementCounter() {  
        counter++;  
    }  
}
```

This use of the word “serialized” has nothing to do with the Serializable interface

We have seen that competing threads could corrupt the “== null” test. However, competing threads could also corrupt the counter since the increment operation counter++ is not atomic.

Therefore, to be sure that the MultiThreadedTest is really producing multiple instances of Singleton, we must make the incrementCounter method synchronized.

Running this test, and witnessing multiple calls to the Singleton constructor once again convinces us that multiple instances are being created.

Main Point 2

- Code that will perform flawlessly in a single-threaded environment may bring disastrous consequences in a multi-threaded environment if shared data is not managed properly.
- In life, a similar situation exists because the particulars of life are always changing. In any domain of life – work, family, health, society – a situation that appears favorable today may become unfavorable tomorrow. How does one prepare for change that cannot be predicted? The only way is to be established in a part of oneself that does not change, no matter how the world changes. That part is first experienced as simple silence in the practice of TM. In time, with regular practice, it becomes firmly established.



CS401 MPP

Java 8 Best Practices

Parallel Processing

Executor

- When many threads need to be managed
 - To create and manage threads properly, Java has an Executor class
 - Starting threads in the way shown before is not optimal
- Two examples of specialized Executor classes:
 - **`Executors.newCachedThreadPool()`** optimized for creation of threads for performing many small tasks or for tasks which involve long wait periods.
 - **`Executors.newFixedThreadPool(numThreads)`** for computationally intensive tasks

Using Executor

- Previous example now modified for **Executor**
 - Plus made **getInstance()** synchronized

```
public class SynchronizedSingleton {  
    private static SynchronizedSingleton instance;  
    public static int counter = 0;  
    private SynchronizedSingleton() {  
        incrementCounter();  
    }  
    synchronized public static SynchronizedSingleton getInstance() {  
        if(instance == null) {  
            instance = new SynchronizedSingleton();  
        }  
        return instance;  
    }  
    private static void incrementCounter() {  
        counter++;  
    }  
}
```

```
public class MultiThreadedTestWithExec {  
    private static Executor exec  
        = Executors.newCachedThreadPool();  
    public static void main(String[] args) {  
        for(int i = 0; i < 10; ++i) {  
            multipleCalls();  
            System.out.println("Num instances: "  
                + SynchronizedSingleton.counter);  
        }  
    }  
    public static void multipleCalls() {  
        Runnable r = () -> {  
            for(int i = 0; i < 500; ++i) {  
                SynchronizedSingleton.getInstance();  
            }  
        };  
        for(int i = 0; i < 100; ++i) {  
            exec.execute(r);  
        }  
        try {  
            Thread.sleep(10);  
        } catch (InterruptedException e) {}  
    }  
}
```

You may notice that the program waits a bit after the last printout. It terminates when the pooled threads have been idle for a while; after some time, the executor terminates them.

Guidelines for ParallelStreams

- When you create a parallel stream from a Collection class in order to process elements in parallel
 - Java handles this request by partitioning the collection and processing each piece with a separate thread
- We will give guidelines for when to use sequential or parallel streams
 - Not every Stream operation, nor every underlying collection type works well with parallel processing

Guidelines

- A) Don't use parallel streams if the number of elements is small
 - the improved performance (if any) will not in this case outweigh the overhead cost of working with parallel streams
- B) Operations that depend on the order of elements in the underlying collection should not be done in parallel
 - For example: `findFirst()`
- C) Expensive terminal operations are extra expensive in parallel (example: `collect(Collectors.joining())`)
 - it will be executed repeatedly in a parallel computation – this could be a good reason to avoid parallel streams in this case.
- D) Translation between primitives and their object wrappers becomes very expensive when done in parallel
 - If you are working with primitives, use the primitive streams, like `IntStream` and `DoubleStream`
- E) Certain data structures can be divided up more efficiently than others
 - `ArrayList`, `HashSet` and `TreeSet` can be partitioned efficiently, but `LinkedLists` cannot
 - (In a `LinkedList`, how can a section of the list be extracted and processed separately?)
- F) Always Benchmark, or at least until you develop a degree of expertise in working with parallel computations
 - it is a good idea to compare the performance of a pipeline executed in parallel with the sequential version

Sample Benchmarks

- Warburton, *Java 8 Lambdas* (p. 84)
 - gives an example of a benchmark test that makes a convincing case for choosing parallel processing over sequential processing for a particular task

```
public int serialArraySum() {  
    return albums.stream()  
        .flatMap(Album::getTracks)  
        .mapToInt(Track::getLength)  
        .sum();  
}
```

```
public int parallelArraySum() {  
    return albums.parallelStream()  
        .flatMap(Album::getTracks)  
        .mapToInt(Track::getLength)  
        .sum();  
}
```

Counts the total number of tracks on all albums

Results

- Warburton reports on a quad core Windows machine:
 - when the number of albums was just 10
 - the sequential version was 8x faster
 - when number of albums was 100
 - the two versions were equally fast
 - when the number of albums was 10,000,
 - the parallel version was 2.5x faster.